

JOCKY: A Cross-Platform Domain-Specific Programming Language for Digital Forensics, Controlled Adversary Emulation, and Security-Control Evaluation

Darshan Ghule

Department of Computer Engineering

Zeal College of Engineering and Research, Pune, India

September 28, 2026

Abstract

Modern endpoint protection platforms combine static signatures, reputation systems, behavioral analytics, process monitoring, memory inspection, kernel telemetry, and network controls. These mechanisms improve security but can complicate legitimate security operations when authorized investigators need to execute custom forensic workloads or conduct controlled adversary-emulation experiments. This paper proposes JOCKY, a cross-platform domain-specific security language and framework designed to express, execute, audit, and evaluate authorized digital-forensic and security-control validation workflows on Windows and Ubuntu.

JOCKY integrates a declarative security language, a platform-independent JOCKY Intermediate Representation (JIR), capability-based authorization, cryptographically verifiable mission packages, cross-platform forensic collectors, controlled program-transformation experiments, centralized orchestration, evidence integrity mechanisms, and telemetry correlation. The architecture separates compilation from endpoint execution and separates experimental variation from unauthorized evasion. This enables researchers to measure whether security controls detect or prevent controlled workloads while preserving forensic evidence and maintaining an auditable chain of custody.

The paper presents the language model, grammar, compiler pipeline, runtime architecture, evidence subsystem, central management plane, experimental methodology, evaluation metrics, security model, limitations, and future work. A reference JOCKY repository is treated only as contextual background; features described here are proposed or experimentally implementable components unless explicitly validated. The framework is intended for government cyber-forensics laboratories, incident-response teams, academic security research, and authorized purple-team exercises. **Keywords:** Digital Forensics, Domain-Specific Language, Cybersecurity, Adversary Emulation, Security-Control Evaluation, Intermediate Representation, Evidence Integrity, Cross-Platform Security, Purple Teaming, CI/CD.

Contents

1	Introduction	2
2	Problem Statement	2
3	Motivation	3
3.1	Forensic Portability	3
3.2	Reproducibility	3
3.3	Security-Control Evaluation	3
3.4	Evidence Integrity	3
4	Research Objectives	3
5	Research Questions	4
6	Threat Model	4
7	Related Work	4
7.1	Digital Forensics	4
7.2	Adversary Emulation	4
7.3	Intermediate Representations	4
7.4	Program Transformation	5
7.5	BYOVD Research	5
7.6	Research Gap	5
8	Proposed System Architecture	5
9	JOCKY Domain-Specific Language	5
9.1	Forensic Mission Example	6
9.2	Controlled Experiment Example	6
10	Language Grammar	7
11	Compiler Design	7
11.1	Semantic Analysis	7
12	JOCKY Intermediate Representation	7
13	Cross-Platform Backend	8
14	Capability-Based Security	8
15	Authorization Model	8

16 Runtime Agent	9
17 Forensic Collection	9
18 Evidence Integrity	9
19 Chain of Custody	10
20 Controlled Transformation Engine	10
21 Offensive-Security Research Layer	10
21.1 Controlled Capability Classes	11
21.2 Offensive Experiment Lifecycle	11
22 Automated Program-Transformation and Polymorphism Research	11
22.1 CI/CD Research Pipeline	12
23 Memory-Execution Security Research	12
24 Kernel and Vulnerable-Driver Security Research	12
25 Living-off-the-Land Research	12
26 Cloud-Mediated Communication Research	13
27 Offensive Research to Forensic Correlation	13
28 Problem-Statement Alignment	13
29 Adversary-Emulation Framework	14
30 Security-Control Evaluation	15
31 Central Management Platform	15
32 Multi-Endpoint Management	15
33 Communication Architecture	15
34 CI/CD Architecture	16
35 Implementation Technology	16
36 Database Design	16
37 Telemetry Correlation	16

38 Experiment Methodology	17
39 Evaluation Metrics	17
40 Transformation Experiment	17
41 Experimental Hypotheses	17
42 Algorithms	18
43 Security Analysis	19
44 Offensive-Security Research Boundary	19
45 Ethical and Legal Considerations	19
46 Expected Results	19
47 Results Template	19
48 Limitations	20
49 Threats to Validity	20
50 Future Work	20
51 Discussion	20
52 Conclusion	20

1 Introduction

Cybersecurity operations increasingly depend on endpoint detection and response (EDR), antivirus, host telemetry, application control, and network monitoring. Earlier security products relied heavily on static signatures, whereas modern platforms combine multiple signals to identify suspicious activity. Process trees, command-line arguments, memory events, module loading, registry changes, network connections, and kernel-level events can all contribute to a detection decision.

For defenders, this evolution introduces an operational question: how can an authorized investigator reproduce a forensic workflow across heterogeneous systems while maintaining evidence integrity and simultaneously evaluate how endpoint controls react to controlled workloads? Conventional scripts can perform individual collection tasks, but they frequently depend on a particular interpreter, operating system, library version, or administrative configuration. They also rarely provide a common representation of the investigation itself.

JOCKY addresses this gap by defining a domain-specific language (DSL) for security investigation and controlled emulation. Instead of treating a security workflow as a collection of unrelated scripts, JOCKY represents the workflow as a structured mission or experiment. The language describes the target platform, required capabilities, collection operations, analysis operations, evidence requirements, observations, reporting requirements, and cleanup actions.

A key design principle is separation of concerns. The compiler translates source language into a platform-independent intermediate representation. A policy engine determines whether requested capabilities are authorized. Platform-specific backends translate the approved representation into supported endpoint operations. Evidence generated by those operations is hashed and associated with a chain-of-custody record. A central server coordinates authorized experiments across multiple endpoints.

The framework also provides a controlled transformation layer for research into program diversity. Rather than defining success as defeating security products, the proposed system treats transformations as experimental variables. A researcher can compare a baseline artifact with semantically equivalent variants and measure changes in detection, execution success, resource consumption, and forensic observability. This converts an offensive-security concept into a measurable security-control evaluation methodology.

The main contributions are: (1) a domain-specific language for declarative forensic missions and controlled security experiments; (2) a versioned JIR for portability; (3) capability-based authorization; (4) Windows and Ubuntu collector abstractions; (5) cryptographically verifiable evidence; (6) centralized orchestration and telemetry correlation; (7) controlled transformation experiments; and (8) a reproducible evaluation methodology.

2 Problem Statement

Modern security solutions can restrict custom executables, scripts, and analysis utilities because their behavior overlaps with techniques used by malicious software. This can affect legitimate forensic operations and controlled security testing.

The research problem is: *How can an authorized security organization use a purpose-built, cross-platform programming language and framework to describe reproducible forensic and adversary-emulation workflows, execute them under explicit authorization, preserve evidence integrity, and systematically evaluate security-control responses?*

The motivating concept includes independent language generation, polymorphic program variation, in-memory execution research, kernel-security research, centralized multi-host management, and cloud-mediated communication. This paper narrows these concepts into a controlled architecture in which sensitive capabilities are gated, experimental variations are logged, and results are measured rather than framed as unrestricted bypass mechanisms.

3 Motivation

3.1 Forensic Portability

Investigators work across Windows and Linux systems. A workflow written for one operating system may require substantial modification for another. JOCKY addresses this through a platform-independent representation and collector interface.

3.2 Reproducibility

Security experiments should be repeatable. If an analyst manually executes a sequence of commands, reproducing the exact experiment later can be difficult. A JOCKY mission records intended operations, authorization context, evidence requirements, and reporting outputs.

3.3 Security-Control Evaluation

Security products should be evaluated using repeatable workloads. JOCKY can define an experiment specifying what is executed, what telemetry is expected, and what constitutes detection.

3.4 Evidence Integrity

A forensic framework must demonstrate that collected evidence was not silently modified. JOCKY associates evidence with cryptographic hashes, timestamps, source endpoints, mission identifiers, and signed metadata.

4 Research Objectives

1. Design a readable security-focused DSL.
2. Develop a parser and semantic-analysis pipeline.
3. Define a stable JIR.
4. Implement Windows and Ubuntu backend abstractions.
5. Implement capability and authorization checks.
6. Develop a central management API.
7. Implement evidence hashing and chain-of-custody records.
8. Build a controlled transformation engine for research experiments.
9. Correlate endpoint telemetry with JOCKY experiment events.
10. Measure portability, forensic yield, detection, latency, and overhead.
11. Demonstrate reproducible multi-endpoint security experiments.

5 Research Questions

RQ1: Can a security DSL express common forensic workflows more reproducibly than ad-hoc scripts?

RQ2: Can JIR permit the same mission specification to execute through Windows and Ubuntu-specific collectors?

RQ3: Can capability-based authorization reduce accidental execution of sensitive operations?

RQ4: Can controlled semantics-preserving transformations generate measurable artifact diversity without changing intended experiment outcomes?

RQ5: Can endpoint telemetry and JOCKY events be correlated into a unified experiment timeline?

RQ6: What performance overhead does the framework introduce?

RQ7: How does controlled artifact variation affect security-control detection in a laboratory environment?

6 Threat Model

The framework is explicitly dual-use. Its assumed users are authorized incident responders, forensic laboratories, government cyber-defense teams, academic researchers, and purple teams. The model considers static signatures, reputation systems, behavioral analytics, process monitoring, memory telemetry, application control, network monitoring, kernel telemetry, and authorization failures.

The endpoint is not assumed completely trustworthy. The runtime must validate signed mission packages, authorization windows, target scope, and capability requirements before execution. Accidental misuse is also considered: if a researcher requests a capability beyond the approved scope, the policy engine rejects it.

7 Related Work

7.1 Digital Forensics

The Sleuth Kit, Autopsy, Volatility, and Velociraptor demonstrate mature approaches to acquisition and artifact analysis. JOCKY complements these systems by focusing on a language-level description of investigations and experiments.

7.2 Adversary Emulation

MITRE ATT&CK provides a common knowledge base for adversary behavior. Atomic Red Team and MITRE Caldera demonstrate repeatable emulation. JOCKY incorporates ATT&CK identifiers as experiment metadata rather than treating them as permission to perform arbitrary actions.

7.3 Intermediate Representations

Compiler research demonstrates the value of intermediate representations for portability, optimization, and transformation. LLVM is a prominent example. JOCKY applies the same

architectural principle to security workflows: JIR represents intended security operations independently of endpoint implementation.

7.4 Program Transformation

Compiler transformations can alter representation while preserving semantics. In JOCKY, transformations are research variables. They are not presented as a guarantee of defeating endpoint controls.

7.5 BYOVD Research

Bring Your Own Vulnerable Driver (BYOVD) is a recognized defensive research topic because vulnerable drivers can provide dangerous kernel capabilities. JOCKY treats driver-related functionality primarily as detection and controlled emulation research. Kernel manipulation is disabled by default and restricted to isolated test environments.

7.6 Research Gap

Existing tools generally specialize in either forensic acquisition, adversary emulation, endpoint telemetry, or compiler transformation. JOCKY proposes a common language and management layer connecting these domains while preserving authorization and evidence integrity.

8 Proposed System Architecture

Figure 1 presents the high-level architecture.

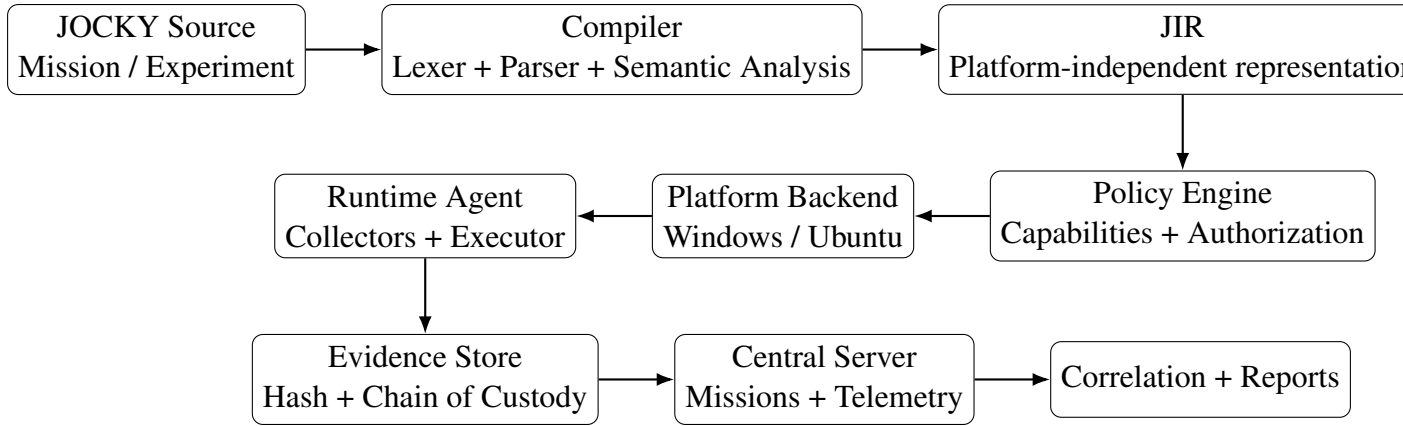


Figure 1: High-level JOCKY architecture.

The architecture contains four planes: language, control, execution, and evidence. The language plane contains the source language, parser, AST, semantic analysis, and JIR. The control plane contains authorization, policy, central management, mission signing, and scheduling. The execution plane contains the endpoint runtime and platform collectors. The evidence plane contains hashing, provenance, telemetry correlation, and reporting.

9 JOCKY Domain-Specific Language

JOCKY is declarative. A program describes what investigation or experiment should occur instead of directly specifying operating-system API calls. Two constructs are provided: `mission`, primarily forensic, and `experiment`, primarily security-control evaluation.

9.1 Forensic Mission Example

```
mission "EndpointInvestigation" {
  target { os = "windows"; architecture = "x64"; }
  requires {
    observe.process;
    observe.network;
    observe.filesystem;
    observe.services;
  }
  collect {
    processes;
    network_connections;
    loaded_modules;
    services;
    startup_items;
    event_metadata;
  }
  analyze {
    process_tree;
    suspicious_modules;
    persistence_artifacts;
  }
  evidence { hash = "SHA256"; chain_of_custody = true; }
  report { timeline; indicators; findings; }
}
```

9.2 Controlled Experiment Example

```
experiment "ProcessMonitoringValidation" {
  objective = "Evaluate endpoint process monitoring";
  technique = "ATTCK_PROCESS_DISCOVERY";
  target = "AUTHORIZED_LAB";
  authorization {
    operator = "team_alpha";
    authorization_id = "AUTH-2026-001";
    scope = "LAB_ONLY";
    expires = "2026-09-27T18:00:00Z";
  }
  requires {
    emulate.process;
    observe.process_events;
    observe.security_events;
  }
  execute { controlled_process_activity; }
  observe { process_events; security_events; endpoint_alerts; }
  evidence { collect_relevant_artifacts; hash = "SHA256";
    chain_of_custody = true; }
  report { detection_status; detection_latency; evidence_completeness
    ; }
}
```

```
cleanup { restore_environment; }
}
```

10 Language Grammar

A simplified grammar is shown below.

```
program = statement+ ;
statement = mission | experiment ;
mission = "mission" STRING "{" mission_block+ "}" ;
experiment = "experiment" STRING "{" experiment_block+ "}" ;
requires_block = "requires" "{" capability+ "}" ;
capability = dotted_name ";" ;
dotted_name = IDENT ( "." IDENT ) * ;
collect_block = "collect" "{" item+ "}" ;
analyze_block = "analyze" "{" item+ "}" ;
observe_block = "observe" "{" item+ "}" ;
execute_block = "execute" "{" item+ "}" ;
evidence_block = "evidence" "{" evidence_item+ "}" ;
report_block = "report" "{" item+ "}" ;
cleanup_block = "cleanup" "{" item+ "}" ;
item = IDENT ";" ;
```

The grammar is intentionally small. Complexity belongs in compiler and runtime semantics rather than syntax.

11 Compiler Design

The compiler pipeline is

Source $\rightarrow$ *AST* $\rightarrow$ *Semantic Analysis* $\rightarrow$ *Policy* $\rightarrow$ *JIR* $\rightarrow$ *Transformation* $\rightarrow$ *Backend*.
(1)

The compiler contains a lexer, parser, AST builder, semantic analyzer, capability validator, policy validator, JIR generator, transformation engine, platform backend, and artifact signer. The compiler must not directly execute forensic actions. It produces a signed, validated mission package.

11.1 Semantic Analysis

Semantic checks verify collector names, target platform, capability declarations, evidence requirements, authorization completeness, and cleanup requirements. The compiler fails closed when validation fails.

12 JOCKY Intermediate Representation

JIR is a versioned platform-independent document.

```
{
  "version": "1.0",
  "kind": "mission",
  "name": "EndpointInvestigation",
```

```

    "id": "MISSION-001",
    "target": {"os": "windows", "architecture": "x64"},
    "capabilities": ["observe.process", "observe.network"],
    "operations": [
        {"type": "collect.processes"},
        {"type": "collect.network_connections"}
    ],
    "evidence": {"hash_algorithm": "SHA256", "chain_of_custody": true}
}

```

JIR permits the same source program to be compiled for multiple operating systems. It should be versioned, schema-validated, and backwards-compatible within defined version ranges.

13 Cross-Platform Backend

JOCKY uses a common collector interface.

```

class ForensicCollector:
    def collect_processes(self): raise NotImplementedError
    def collect_network(self): raise NotImplementedError
    def collect_modules(self): raise NotImplementedError
    def collect_services(self): raise NotImplementedError
    def collect_autostart(self): raise NotImplementedError

```

Table 1: Cross-platform collector abstraction

Logical operation	Windows example	Ubuntu example
Process collection	Native process metadata	/proc process metadata
Network collection	Native socket/network interfaces	/proc/net and netlink
Services	Service Control Manager	systemd metadata
Modules	Loaded module metadata	/proc/ <i>i</i> pid/ <i>i</i> /maps
Startup artifacts	Registry/startup locations	systemd/cron/autostart artifacts
Event metadata	Windows event infrastructure	journald/audit metadata

The framework records provenance for every collector result.

14 Capability-Based Security

Capabilities prevent arbitrary operations from being requested. Examples include `observe.process`, `observe.network`, `observe.filesystem`, `observe.events`, `observe.services`, `observe.memory_metadata`, `emulate.process`, `emulate.execution`, `evidence.write`, and `report.timeline`. Each capability is evaluated against source declaration, operator role, mission authorization, endpoint policy, time window, and target scope. Sensitive emulation capabilities are disabled by default.

15 Authorization Model

Authorization is represented as a signed mission policy.

```
{
  "authorization_id": "AUTH-2026-001",
  "operator_id": "team_alpha",
  "scope": "LAB_ONLY",
  "allowed_endpoints": ["WIN-LAB-01", "UBUNTU-LAB-01"],
  "allowed_capabilities": ["observe.process", "observe.network", "
    emulate.process"],
  "expires": "2026-09-27T18:00:00Z"
}
```

The execution decision is:

$$Execute = AuthValid \wedge ScopeValid \wedge TimeValid \wedge CapabilityValid \wedge SignatureValid. \quad (2)$$

If any condition is false, execution is rejected.

16 Runtime Agent

The runtime agent performs mission verification, collector execution, evidence creation, and telemetry reporting. It should use least privilege and avoid automatically obtaining unrestricted administrator or root privileges. Its lifecycle is: register endpoint, receive signed mission, verify signature, validate scope, validate capabilities, execute approved operations, hash evidence, send telemetry, generate report, and perform cleanup.

17 Forensic Collection

The initial collector set focuses on high-value artifacts. Process records may include PID, parent PID, process name, executable path, start time, user identity, and permitted command-line metadata. Network records may include local address, local port, remote address, remote port, protocol, and owning process where available. Persistence collection inspects authorized persistence locations and reports metadata rather than modifying them. Module collection records names, paths, hashes where feasible, and associated processes.

18 Evidence Integrity

For each evidence object E , JOCKY calculates

$$H(E) = SHA256(E). \quad (3)$$

The evidence record contains evidence identifier, mission identifier, endpoint identifier, collection timestamp, collector version, hash algorithm, evidence hash, previous chain record, and signature metadata. A chain record can be represented as

$$C_i = H(E_i \| C_{i-1} \| T_i), \quad (4)$$

where T_i is the collection timestamp. This permits later verification of evidence sequence integrity.

19 Chain of Custody

```
{
  "evidence_id": "EV-0001",
  "mission_id": "MISSION-001",
  "endpoint": "WIN-LAB-01",
  "collector": "process_collector",
  "timestamp": "2026-09-27T10:21:04Z",
  "hash_algorithm": "SHA256",
  "hash": "...",
  "operator": "team_alpha"
}
```

The central server preserves the original record and restricts modification through role-based controls.

20 Controlled Transformation Engine

The research version includes a transformation engine to study how structurally different but semantically equivalent artifacts interact with security controls. Safe research variables include identifier renaming, metadata variation, non-functional code-layout changes, deterministic versus randomized build parameters, resource-layout variation, and equivalent control-flow representations in a laboratory compiler.

Let P be a baseline program and T_i a semantics-preserving transformation:

$$P_i = T_i(P). \quad (5)$$

The experiment tests whether

$$Behavior(P) \approx Behavior(P_i) \quad (6)$$

while comparing detection outcomes. The framework does not promise or require evasion.

21 Offensive-Security Research Layer

JOCKY is intentionally designed to support an offensive-security research workflow while maintaining a strict distinction between authorized adversary emulation and unrestricted exploitation. The offensive layer treats security behaviour as a typed experiment rather than as an unconstrained payload-generation mechanism. Each experiment contains an objective, technique metadata, authorization context, target allowlist, required capabilities, expected telemetry, evidence requirements, time limit, and cleanup procedure.

This design directly addresses the language-oriented requirement of the research problem. Instead of writing separate PowerShell, Bash, Python, or C/C++ utilities for each platform, the operator describes the intended security behaviour in JOCKY. The compiler converts that description into JIR, validates its capabilities and authorization, and then invokes the appropriate platform backend. Consequently, offensive intent becomes a first-class programming-language abstraction while execution remains policy constrained.

21.1 Controlled Capability Classes

Capabilities are divided into observation, emulation, evidence, and management classes. Observation capabilities include process, network, service, file, event, and memory-metadata collection. Emulation capabilities represent controlled behaviours that are useful for security-control validation. Sensitive capabilities are disabled by default and require explicit authorization. An ATT&CK technique identifier is metadata and never grants permission to execute an operation.

21.2 Offensive Experiment Lifecycle

The offensive experiment lifecycle is:

1. Define the experiment and research objective.
2. Declare the target environment and required capabilities.
3. Validate the authorization manifest and endpoint allowlist.
4. Compile the experiment into JIR.
5. Apply approved, semantics-preserving research transformations where required.
6. Sign and register the experiment package.
7. Execute only on authorized laboratory endpoints.
8. Collect endpoint and security-control telemetry.
9. Correlate observations with JOCKY events and evidence.
10. Restore the laboratory state and generate the research report.

22 Automated Program-Transformation and Polymorphism Research

The original research motivation includes automated polymorphic generation. In JOCKY this requirement is implemented as a controlled compiler-transformation research layer rather than as an unrestricted malware-polymorphism engine. The purpose is to investigate whether semantics-preserving representation changes influence security-control observations.

A baseline program P can be passed through approved transformations $T_1, T_2, \dots, T_n$ to generate a family of variants:

$$V = P, T_1(P), T_2(P), \dots, T_n(P). \quad (7)$$

Each variant receives a unique experiment identifier, source revision, compiler version, transformation configuration, artifact hash, target platform, and authorization reference. The experimental system can therefore compare source-equivalent variants without losing provenance.

Research variables may include identifier representation, metadata layout, build configuration, resource organization, and other semantics-preserving compiler transformations. The study records whether the transformation changes execution correctness, resource consumption, telemetry volume, detection outcome, or detection latency. No claim is made that any transformation defeats a particular product.

22.1 CI/CD Research Pipeline

The CI/CD pipeline consists of source validation, unit tests, semantic validation, authorization validation, JIR generation, approved transformation, platform compilation, artifact signing, registry storage, laboratory deployment, telemetry collection, and report generation. Every build is reproducible from its recorded inputs. The pipeline therefore serves as both a software-engineering mechanism and an experimental-control mechanism.

23 Memory-Execution Security Research

Memory-oriented execution is relevant because modern endpoint products inspect process memory, module loading, execution transitions, and related telemetry. JOCKY therefore defines a memory-execution research capability for isolated laboratory experiments. The capability is designed to measure visibility rather than to provide unrestricted injection or evasion functionality. Experiments may compare a benign baseline workload with a controlled memory-oriented workload and record process events, memory-related telemetry, module observations, security alerts, execution time, and evidence completeness. Platform-specific implementations are kept behind the JOCKY backend interface so that the language semantics remain stable.

The research outcome is a measurable comparison of prevention, detection, observability, and latency. Any technique that requires intrusive process manipulation is disabled by default and confined to an isolated test environment with explicit authorization.

24 Kernel and Vulnerable-Driver Security Research

Kernel-level security is another important dimension of the problem because security products may use kernel telemetry and callbacks. JOCKY therefore provides a controlled kernel-security research module for studying vulnerable-driver exposure and defensive response.

The module can maintain a catalog of driver identifiers, affected platform versions, vulnerability metadata, expected security events, and laboratory test status. Experiments focus on detection, prevention, telemetry quality, and evidence collection. Arbitrary kernel modification and disabling of third-party security controls are outside the normal runtime contract.

This distinction is important for reproducibility and safety: the research evaluates whether security controls recognize risky driver conditions rather than defining success as silently disabling those controls.

25 Living-off-the-Land Research

JOCKY includes a LOTL research abstraction because legitimate operating-system utilities can be involved in both administrative workflows and adversary behaviour. A JOCKY experiment identifies the intended behaviour and records the relevant process, parent-child relationship, command metadata, timestamps, and security telemetry.

The same experiment can therefore be represented consistently across platforms while allowing each backend to select the appropriate laboratory implementation. The system measures whether the security stack detects, prevents, or records the behaviour and how quickly the event becomes visible to analysts.

26 Cloud-Mediated Communication Research

The original problem motivation includes communication through trusted cloud infrastructure and CDN services. JOCKY addresses this as a network-security research problem rather than as a covert command-and-control feature. The prototype uses authenticated TLS communication and may use legitimate cloud APIs or approved relay infrastructure when permitted by the laboratory.

Experiments can compare direct communication with cloud-mediated communication and measure network telemetry, proxy visibility, authentication behaviour, latency, packet or request metadata, and security alerts. Any domain-fronting-related research is treated as a separately authorized laboratory experiment and is not a default operational capability of the runtime.

This approach preserves the research value of the original requirement while avoiding the assumption that traffic must be hidden from legitimate security monitoring.

27 Offensive Research to Forensic Correlation

A central contribution of JOCKY is the ability to connect offensive experiments with forensic evidence. For every experiment, the management plane associates the action timestamp, endpoint, operator, JOCKY source hash, JIR hash, artifact identifier, transformation configuration, endpoint telemetry, security alert, collected evidence, and cleanup status.

This creates a unified timeline in which a researcher can ask not only whether an experiment was detected, but also which forensic artifacts were generated and whether those artifacts were sufficient to reconstruct the event. The resulting dataset can support defensive engineering, incident-response training, and academic analysis.

28 Problem-Statement Alignment

Table 2 summarizes how the final JOCKY design addresses the major requirements of the motivating problem statement.

Table 2: JOCKY alignment with the motivating requirements

Requirement	JOCKY response
Independent programming language	Dedicated JOCKY DSL with lexer, parser, semantic model, and security-oriented primitives.
Cross-platform compiler	Platform-independent JIR with Windows and Ubuntu/Linux backends.
Program transformation	Controlled semantics-preserving transformation layer integrated with CI/CD experiments.
Forensic analysis	Process, network, module, service, persistence, event, and evidence collectors.
Memory-oriented research	Isolated memory-execution experiments with telemetry and evidence correlation.
Kernel/BYOVD research	Vulnerable-driver exposure and defensive-response research with sensitive capabilities disabled by default.
LOTL research	Controlled trusted-utility experiments with process and telemetry correlation.
Central management	Mission scheduling, endpoint registration, artifact management, evidence ingestion, and reporting.
Multi-system operation	Same JIR mission can be dispatched to multiple authorized endpoints with endpoint-specific results.
Cloud/CDN research	Authenticated cloud-mediated communication experiments and visibility measurement.
Security-control evaluation	Detection, prevention, latency, telemetry quality, and forensic-yield metrics.
Governance	Signed packages, capability checks, target allowlists, short-lived authorization, audit logs, and cleanup.

29 Adversary-Emulation Framework

JOCKY supports controlled emulation as an experiment type. Each experiment contains an objective, technique identifier, authorization, target scope, required capabilities, controlled action, observation requirements, evidence requirements, and cleanup requirements. ATT&CK

identifiers are metadata. They do not grant execution permission.

30 Security-Control Evaluation

A detection event can contain event identifier, mission identifier, endpoint, source, event time, severity, and technique metadata. Detection latency is

$$L_d = T_{\text{detection}} - T_{\text{action}}. \quad (8)$$

The framework should evaluate prevention, detection, telemetry quality, latency, and analyst usability rather than only a binary bypass result.

31 Central Management Platform

The central server provides authentication, operator management, endpoint registration, mission creation, mission signing, experiment scheduling, evidence ingestion, telemetry correlation, and reporting. A FastAPI prototype may expose:

```
POST /auth/login
POST /missions
GET /missions/{id}
POST /missions/{id}/sign
POST /missions/{id}/dispatch
GET /endpoints
GET /evidence/{id}
GET /experiments/{id}/timeline
GET /reports/{id}
```

All privileged endpoints require authentication and authorization.

32 Multi-Endpoint Management

For endpoint set $E = \{e_1, e_2, \dots, e_n\}$, the server dispatches the same JIR package while preserving endpoint-specific records. The experiment result is

$$R = \{R_1, R_2, \dots, R_n\}. \quad (9)$$

This enables comparison of operating systems, security configurations, agent versions, and resource usage.

33 Communication Architecture

The prototype uses authenticated HTTPS/TLS. Communication is separated into control messages, evidence transfer, telemetry, and management requests. Cloud infrastructure may be used as an ordinary authenticated transport layer when permitted by the laboratory. The design emphasizes TLS, endpoint authentication, short-lived credentials, replay protection, and audit logging rather than covert communications.

34 CI/CD Architecture

CI/CD makes experiments reproducible. The pipeline is: source, syntax and unit tests, policy validation, build and controlled transformation, signing, artifact registry, and authorized laboratory deployment. Each build stores source version, compiler version, transformation configuration, target platform, artifact hash, authorization reference, and timestamp.

35 Implementation Technology

Table 3: Recommended implementation stack

Component	Technology
DSL parser	Python + Lark
Schemas	Pydantic
Compiler prototype	Python
Management API	FastAPI + Uvicorn
Database	SQLite for prototype; PostgreSQL for deployment
Cryptography	Python cryptography package
Windows collectors	Native Windows interfaces
Ubuntu collectors	/proc and system interfaces
Testing	pytest
Containerization	Docker
Visualization	React or server-rendered dashboard
Virtualization	VirtualBox, VMware, KVM, or QEMU

36 Database Design

The central database contains users, roles, endpoints, missions, experiments, artifacts, evidence, telemetry, detection events, and audit records. A simplified model is:

```
USERS(user_id, username, role_id)
ENDPOINTS(endpoint_id, hostname, os, architecture, agent_version,
           status)
MISSIONS(mission_id, name, type, source_hash, status,
         authorization_id)
EVIDENCE(evidence_id, mission_id, endpoint_id, collector, hash,
         timestamp)
TELEMETRY(event_id, mission_id, endpoint_id, source, timestamp,
          event_type)
AUDIT_LOG(audit_id, operator_id, action, timestamp, result)
```

37 Telemetry Correlation

The central engine merges JOCKY execution events, endpoint telemetry, security alerts, evidence events, and operator actions. A unified event can be represented as

$$U_i = (t_i, s_i, e_i, m_i, p_i), \quad (10)$$

where t_i is time, s_i is source, e_i is event type, m_i is mission identifier, and p_i is endpoint identifier. Events are grouped into timelines using mission, endpoint, and time-window relationships.

38 Experiment Methodology

Experiments use isolated virtual machines. At minimum the testbed contains a Windows laboratory VM, Ubuntu laboratory VM, JOCKY management server, test database, and telemetry infrastructure. Production systems are excluded.

The baseline condition runs the equivalent forensic workload using a conventional implementation. Metrics include execution time, CPU usage, memory usage, evidence count, and evidence integrity. The JOCKY condition runs the same logical workflow through the framework. The difference is

$$\Delta T = T_{JOCKY} - T_{baseline}. \quad (11)$$

Resource overhead is

$$Overhead = \frac{Resource_{JOCKY} - Resource_{baseline}}{Resource_{baseline}} \times 100. \quad (12)$$

39 Evaluation Metrics

Portability is

$$P = \frac{Successful\ Platforms}{Targeted\ Platforms}. \quad (13)$$

Forensic yield is

$$F = \frac{Valid\ Evidence\ Objects}{Expected\ Evidence\ Objects}. \quad (14)$$

Integrity is

$$I = \frac{Verified\ Evidence\ Objects}{Collected\ Evidence\ Objects}. \quad (15)$$

Detection rate is

$$DR = \frac{Detected\ Experiments}{Total\ Experiments}. \quad (16)$$

False-positive rate is

$$FPR = \frac{False\ Alerts}{Benign\ Experiments}. \quad (17)$$

40 Transformation Experiment

To evaluate controlled program variation, construct $V = \{v_0, v_1, \dots, v_n\}$ where v_0 is the baseline and each v_i is produced using an approved transformation configuration. Each variant records artifact hash, source version, transformation configuration, execution result, detection result, detection latency, and resource consumption. The research question is whether detection behavior changes systematically with transformation class.

41 Experimental Hypotheses

H1: JOCKY missions can be reused across supported operating systems through platform-specific collectors.

H2: JOCKY produces more consistent experiment records than manually executed workflows.

H3: Cryptographic evidence verification detects intentional modification of stored evidence.

H4: Controlled transformations produce measurable structural differences while preserving intended experiment semantics.

H5: Centralized correlation improves association between experiments, endpoint telemetry, and alerts.

42 Algorithms

Algorithm 1 JOCKY Mission Validation

Require: Mission M , policy P , authorization A

Ensure: Accept or Reject

```
1: Parse  $M$ 
2: if syntax invalid then
3:   return Reject
4: end if
5: Build AST
6: Validate semantic constraints
7: Extract requested capabilities  $C$ 
8: if  $C$  not subset of  $P.allowedCapabilities$  then
9:   return Reject
10: end if
11: if  $A$  invalid then
12:   return Reject
13: end if
14: if scope not contained in  $A.scope$  then
15:   return Reject
16: end if
17: Generate JIR
18: Sign package
19: return Accept
```

Algorithm 2 Evidence Verification

Require: Evidence object E , recorded hash H_r

```
1: Compute  $H_c \leftarrow SHA256(E)$ 
2: if  $H_c = H_r$  then
3:   return Valid
4: else
5:   return IntegrityFailure
6: end if
```

43 Security Analysis

JOCKY itself must be treated as sensitive security software. A fail-closed policy rejects missing authorization. Least privilege limits collectors to required permissions. Signed packages prevent unauthorized mission modification. Short-lived authorization limits the window of execution. Auditability ensures privileged actions are recorded. Isolation keeps sensitive experiments away from production networks.

44 Offensive-Security Research Boundary

The motivating research includes advanced offensive-security concepts because defensive systems must be evaluated against realistic behavior. The framework distinguishes controlled emulation from operational exploitation. Emulation reproduces a behavior for measurement in an authorized laboratory; operational exploitation is unrestricted exploitation or evasion against real systems and is outside the proposed implementation.

Kernel-security research may include detection and controlled simulation of vulnerable-driver conditions without distributing a mechanism intended to disable third-party security products on arbitrary machines. Similarly, memory-execution research can use isolated synthetic workloads rather than an unrestricted injection framework.

45 Ethical and Legal Considerations

The framework should require written authorization, explicit target scope, experiment expiration, operator identity, audit records, evidence integrity, laboratory isolation for sensitive tests, and cleanup procedures. JOCKY should never be presented as a universal antivirus-bypass platform. Its research objective is to measure and improve security controls.

46 Expected Results

Expected outputs include a working parser/compiler prototype, JIR generation, cross-platform collection through common abstractions, verifiable evidence hashes, centralized mission management, reproducible experiments, correlated endpoint telemetry, quantifiable runtime overhead, detection latency, and measurable artifact diversity under controlled transformations. The final study must report measured values rather than invented results.

47 Results Template

Table 4: Experimental results template

Metric	Baseline	JOCKY	Difference	Unit
Execution time	—	—	—	ms
CPU usage	—	—	—	%
Memory usage	—	—	—	MB
Evidence objects	—	—	—	count
Evidence integrity	—	—	—	%
Detection rate	—	—	—	%
Detection latency	—	—	—	ms

48 Limitations

A prototype cannot represent every Windows and Linux forensic source. Endpoint security products differ substantially between vendors and configurations. Telemetry availability varies by operating system and policy. A laboratory experiment cannot fully represent a production enterprise. The transformation engine is intentionally constrained and designed to measure structural diversity rather than provide unrestricted security-control bypass functionality. No security framework can guarantee invisibility against all endpoint controls; JOCKY therefore measures observability and detection rather than making absolute evasion claims.

49 Threats to Validity

Internal validity may be affected by endpoint configuration changes. Experiments should control variables and repeat runs. External validity is limited because one EDR product or one operating-system configuration may not generalize. Construct validity requires more than detection rate; prevention, telemetry quality, latency, evidence completeness, and analyst usability should also be measured. Reproducibility requires recording compiler version, configuration, mission hash, endpoint configuration, and transformation parameters.

50 Future Work

Future work includes additional operating-system backends, formal verification of JIR transformations, forensic timeline generation, evidence provenance graphs, richer telemetry correlation, ATT&CK integration, hardware-backed signing, distributed experiment scheduling, statistical analysis of security-control behavior, formal policy verification, a memory-safe runtime, and integration with forensic-analysis platforms.

51 Discussion

The principal value of JOCKY is not merely the creation of another scripting language. Its value lies in treating security investigations as reproducible programs with explicit authorization, evidence, and measurement semantics. Traditional scripts answer what commands should be executed; JOCKY describes what security experiment or forensic mission is authorized, what capabilities it requires, what evidence it should produce, how it should be observed, and how it can be reproduced.

The transformation engine is particularly relevant to security research. Defenders cannot evaluate robustness using only one static artifact. Controlled variation allows researchers to investigate whether detection mechanisms depend on superficial properties or deeper behavioral signals. Central orchestration is similarly valuable because the same experiment can be executed on multiple controlled endpoints and compared systematically.

52 Conclusion

This paper presented JOCKY, a cross-platform domain-specific security language and framework for digital forensics, controlled adversary emulation, and security-control evaluation. The proposed system combines a declarative DSL, compiler frontend, JIR, capability-based security, cross-platform collectors, cryptographic evidence integrity, centralized orchestration, telemetry correlation, and controlled program transformation.

The architecture intentionally avoids defining success as unrestricted antivirus or EDR bypass. Instead, it establishes a measurable research framework in which security-control behavior can be evaluated under authorized and reproducible conditions. The framework is applicable to government cyber-defense laboratories, forensic investigation teams, academic cybersecurity research, and purple-team exercises. The next stage is implementation and controlled experimental validation; actual performance, detection, and portability results should only be reported after testing.

Acknowledgment

The author acknowledges the open-source cybersecurity, digital-forensics, and compiler communities whose research and software ecosystems provide foundations for the proposed work.

References

- [1] MITRE, “MITRE ATT&CK,” MITRE Corporation. Available: <https://attack.mitre.org/>
- [2] Red Canary, “Atomic Red Team,” Available: <https://github.com/redcanaryco/atomic-red-team>
- [3] MITRE, “MITRE Caldera,” Available: <https://github.com/mitre/caldera>
- [4] C. Lattner and V. Adve, “LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation,” Proceedings of CGO, 2004.
- [5] Volatility Foundation, “The Volatility Framework,” Available: <https://www.volatilityfoundation.org/>
- [6] The Sleuth Kit, “Open Source Digital Forensics,” Available: <https://www.sleuthkit.org/>
- [7] Basis Technology, “Autopsy Digital Forensics Platform,” Available: <https://www.autopsy.com/>
- [8] Velocidex, “Velociraptor Endpoint Visibility and Collection,” Available: <https://docs.velociraptor.app/>
- [9] P. Cichonski et al., “Computer Security Incident Handling Guide,” NIST SP 800-61.
- [10] K. Kent et al., “Guide to Integrating Forensic Techniques into Incident Response,” NIST SP 800-86.
- [11] NIST, “Security and Privacy Controls for Information Systems and Organizations,” NIST SP 800-53.
- [12] ISO/IEC, “ISO/IEC 27037: Guidelines for identification, collection, acquisition and preservation of digital evidence.”
bibitemcaldera2 MITRE, “MITRE Caldera Documentation,” MITRE Corporation.
bibitemnistred NIST, “Guide to Cybersecurity Event Recovery,” NIST Special

Publication 800-184.

bibitemloldrivers LOLDrivers Project, “Living Off The Land Drivers,” defensive research resource.

- [13] I. You and K. Yim, “Malware Obfuscation Techniques: A Brief Survey,” BWCCA, 2010.
- [14] M. Egele, T. Scholte, E. Kirda, and C. Kruegel, “A Survey on Automated Dynamic Malware-Analysis Techniques and Tools,” ACM Computing Surveys, 2012.
- [15] Microsoft, “Sysmon,” Microsoft Sysinternals. Available: <https://learn.microsoft.com/sysinternals/downloads/sysmon>
- [16] Python Software Foundation, “Python Documentation.” Available: <https://docs.python.org/>
- [17] FastAPI, “FastAPI Documentation.” Available: <https://fastapi.tiangolo.com/>
- [18] Pydantic, “Pydantic Documentation.” Available: <https://docs.pydantic.dev/>
- [19] Lark Parser, “Lark Documentation.” Available: <https://lark-parser.readthedocs.io/>
- [20] srnsksyp, “JOCKY,” GitHub repository, reference material. Available: <https://github.com/srnsksyp/JOCKY>